

Experiment 3: Hardware Implementation of Euclid's GCD Algorithm Using Verilog

Objective

The objectives of this experiment are:

- To design and implement Euclid's algorithm for finding the greatest common factor (GCD) of two natural numbers.
- To analyze different RTL design styles for GCD computation using enable registers and multiplexer-based architectures.

Background

The computation of the Greatest Common Divisor (GCD) is essential in various applications, including cryptography, signal processing, and digital logic design. Euclid's algorithm is one of the oldest and most efficient methods for computing the GCD. It iteratively reduces the problem size by using the relation:

$$\text{gcd}(A, B) = \text{gcd}(A - B, B), \quad \text{for } A > B, \quad (1)$$

which is applied repeatedly until B becomes zero, at which point A represents the GCD. Figure 1 shows the high-level block diagram of the GCD architecture.

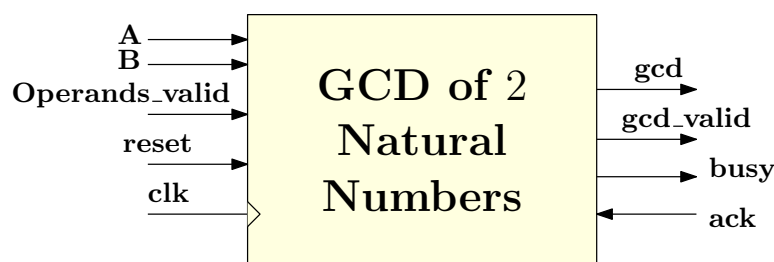


Figure 1: High-level block diagram of the GCD architecture

Experimental Procedure

1. Design a clear and legible datapath and control architecture based on design questions 1, 2 and 3, including all required signals.
2. Implement and verify the designs using parameterized Verilog HDL. The combinational logic and sequential logic shall be clearly separated in the Verilog design.

Design Questions

1. Design and implement a Verilog HDL module to compute the GCD of two natural numbers A and B using enable registers. The design shall use a 3-state FSM (IDLE, BUSY, DONE) with an acknowledge (ACK) signal from a downstream module. The design shall assert completion in the Done state and wait for ACK before returning to the IDLE state.
2. Design and implement an alternative Verilog HDL module for GCD computation by replacing enable registers with 4×1 multiplexers at the inputs of registers A and B . The design shall use the same 3-state FSM with the ACK signal.
3. Design and implement a Verilog HDL module to handle the case where inputs A and B do not arrive simultaneously. In this case, a single Operands_valid signal shall not be used; instead, separate A_valid and B_valid signals shall be employed to qualify the inputs before initiating the GCD computation. The design shall use a 5-state FSM to handle independent arrival of operands (IDLE, A_WAIT, B_WAIT, BUSY, DONE).

The design (question 3) may be implemented using either enable registers or a multiplexer-based datapath.